

南開大學
Nankai University

计算机学院

并行程序设计 **Lab3** 报告

Pthread/OpenMP 多线程编程实验

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 5 月 24 日

项目仓库：https://github.com/Aurora1094/NKU-ParallelProgramming

摘　要

本实验选择近似最近邻搜索（Approximate Nearest Neighbor Search, ANN）作为并行优化对象，围绕 DEEP100K 数据集上的高维向量查询任务，在前序 SIMD 优化基础上进一步实现并比较 Pthread 与 OpenMP 两类多线程并行方法。实验覆盖 Flat-SIMD、SQ-SIMD、PQ-SIMD、IVF-SIMD、IVF-PQ-SIMD 和 HNSW 等多种搜索方法，并在 ARM/openEuler 与 x86/WSL2 平台上进行性能测试、参数分析与跨平台对比。

本文主要采用 query-level parallelism 作为并行策略，将批量 query 划分给多个线程独立处理。Pthread 版本通过显式创建线程和划分 query 区间实现并行，OpenMP 版本则使用 `parallel for` 并行化 query 循环，并进一步比较了 `static`、`dynamic` 和 `guided` 等调度策略。实验结果表明，在相同算法参数下，Pthread 与 OpenMP 不改变单个 query 的搜索逻辑，因此 Recall 基本保持一致，性能差异主要体现在 wall latency、线程调度和负载均衡上。

综合性能测试结果表明，算法结构对查询性能的影响显著大于线程 API 本身。在 x86 平台 14 组方法中，HNSW OpenMP 版本取得最低查询延迟，wall latency 为 $15.77\,\mu s$，Recall@10 达到 1.0；IVF-PQ-SIMD OpenMP 版本在 Recall@10 为 0.998850 的情况下达到 $19.59\,\mu s$，表现出较好的精度–效率折中。相比之下，Flat-SIMD 和 SQ-SIMD 虽然使用 SIMD 加速距离计算，但由于仍需扫描较多候选，查询延迟相对较高。

本文还进一步开展了参数敏感性分析。对于 IVF，*nprobe* 是影响 Recall 与 latency 的关键参数，增大 *nprobe* 可以提高搜索覆盖率，但也会增加候选扫描开销；对于 IVF-PQ，$rerank_p$ 控制进入 float32 精排的候选数量，是决定 recall-latency trade-off 的核心参数；对于 HNSW，$efSearch$ 增大会提高搜索稳定性，但也会带来更高的图遍历和候选队列维护开销。此外，perf profiling 和 ARM/x86 跨平台实验表明，x86 平台在多数方法上明显快于 ARM 平台，但不同算法的加速幅度并不一致，说明 ANN 查询性能还受到 SIMD 指令集、缓存层次、访存模式和算法结构的共同影响。

综上，本实验表明，在 ANN 查询任务中，单纯更换线程 API 并不是性能优化的核心；算法结构、候选规模、参数选择、调度策略和平台体系结构共同决定最终性能。HNSW 在查询阶段具有最低延迟，但索引构建成本较高；IVF-PQ-SIMD 在高 Recall 和低 latency 之间具有较好折中，是本文实验中综合性能较优的主线方案。

关键词：近似最近邻搜索；Pthread；OpenMP；IVF-PQ；HNSW

目录

1 前言 **1**
1.1 实验环境 . . . 1
1.2 实验内容概述 . . . 1
1.3 实验数据与评价指标 . . . 2

2 算法实现原理与并行优化 **2**
2.1 ANN 基础搜索与 SIMD 优化 . . . 2
2.2 IVF（Inverted File） . . . 3
2.3 IVF-PQ（Inverted File + Product Quantization） . . . 4
2.4 HNSW 图索引方法 . . . 5
2.5 多线程并行优化 . . . 6

3 实验设计 **7**
3.1 Flat-SIMD 多线程实验 . . . 8
3.2 SQ-SIMD 多线程实验 . . . 8
3.3 PQ-SIMD 多线程实验 . . . 8
3.4 IVF-SIMD 多线程实验 . . . 9
3.5 IVF-PQ-SIMD 多线程实验 . . . 9
3.6 HNSW 多线程实验 . . . 9
3.7 参数敏感性与跨平台实验设计 . . . 9

4 实验结果与分析 **9**
4.1 不同算法的综合性能对比 . . . 9
4.2 Pthread 与 OpenMP 并行效果对比 . . . 11
4.3 参数敏感性分析 . . . 11
4.3.1 IVF 参数敏感性 . . . 11
4.3.2 IVF-PQ 参数敏感性 . . . 13
4.3.3 HNSW 参数敏感性 . . . 14
4.4 OpenMP 调度策略分析 . . . 16
4.5 Profiling 与体系结构分析 . . . 17
4.6 ARM 与 x86 跨平台对比分析 . . . 17

5 总结 **19**
5.1 实验结论 . . . 19
5.2 不足与改进方向 . . . 19
5.3 个人心得体会 . . . 19

A 线程扩展性实验数据 **20**

B 原始实验数据 **21**

1 前言

1.1 实验环境

本实验主要在 ARM/openEuler 服务器平台完成，并在 x86/WSL2 本地平台上进行补充对比。两类平台的软硬件环境如表 1 所示。

表 1: 实验软硬件环境

项目	ARM/openEuler 服务器	x86/WSL2 本地平台
CPU 架构	aarch64	x86_64
CPU 型号	Kunpeng-920	Intel Core i9-14900HX
核心/线程	8 cores	32 logical CPUs
操作系统	openEuler 22.03 LTS-SP4	Ubuntu 22.04 WSL2
编译器	g++ 10.3.1	g++ 11.4.0
SIMD 指令集	NEON / ASIMD	AVX2 / FMA
内存	30 GiB	15GiB
测试线程数	1/2/4/8	8

1.2 实验内容概述

本实验围绕 DEEP100K 数据集上的近似最近邻搜索（Approximate Nearest Neighbor Search, ANN）展开。在前序 SIMD 优化基础上[1]进一步实现并比较 Pthread 与 OpenMP 两类多线程并行方法。

实验主要包含以下几个部分：

1. 在 Flat-SIMD、SQ-SIMD、PQ-SIMD、IVF-SIMD、IVF-PQ-SIMD 和 HNSW 等算法基础上实现多线程查询，并分别构造 Pthread 与 OpenMP 版本。
2. 对不同算法的查询延迟、召回率和并行加速效果进行比较，分析不同算法结构对多线程优化收益的影响。
3. 针对 IVF、IVF-PQ 和 HNSW 等索引算法开展参数敏感性实验，分析 `nlist`、`nprobe`、`rerank_p`、`efSearch` 等参数对 recall-latency trade-off 的影响。
4. 对 OpenMP 的不同调度策略进行测试，比较 `static`、`dynamic` 和 `guided` 等 schedule 方式在 IVF-SIMD 上的性能差异。
5. 结合 perf profiling 结果，从 IPC、cache miss、branch miss 和 full-program elapsed time 等角度解释不同算法版本的底层性能差异。
6. 在 ARM/openEuler 与 x86/WSL2 两个平台上进行对比实验，分析不同体系结构和 SIMD 指令集对 ANN 查询性能的影响。

整体而言，本文的实验主线是：先通过 SIMD 加速单次距离计算，再通过 IVF/PQ/SQ/HNSW 等算法结构减少候选规模或降低距离计算成本，最后通过 Pthread 与 OpenMP 在 query 维度上提升并行吞吐。通过这一过程，可以系统分析算法结构、线程接口、参数选择和硬件平台对 ANN 查询性能的综合影响。

[1]本文中的 Flat-SIMD、SQ-SIMD 和 PQ-SIMD 等向量化版本主要继承自 lab2-SIMD 实验；本实验重点在于在这些算法外层加入 Pthread/OpenMP query-level 并行，并分析并行效果。

1.3 实验数据与评价指标

本文统一使用 DEEP100K 作为实验数据集。该数据集包含 base 向量、query 向量和 ground truth 文件。其中 base 集合包含 100000 个 96 维 float32 向量，query 集合包含 10000 个 96 维 float32 查询向量，ground truth 文件提供每个查询对应的真实近邻结果。实验任务是在 base 集合中为每个 query 返回近似最近邻，并与 ground truth 进行比较。

本文主要采用以下指标评价算法性能：

1. **Recall@10**：用于衡量近似最近邻搜索的准确性。Recall 越高，说明返回结果与真实近邻集合的重合程度越高。
2. **average latency**：程序内部统计的平均查询延迟，单位为 μs。该指标反映单个 query 在算法内部的平均处理时间。
3. **wall latency**：使用 wall-clock time 统计的 query-only 平均查询延迟，单位为 μs。其计算方式为查询阶段总墙钟时间除以 query 数量。由于该指标包含多线程执行时的实际端到端开销，因此更适合用于比较 Pthread 与 OpenMP 的查询性能。
4. **total wall time**：查询阶段的总墙钟时间，单位为 μs。该指标用于观察一组 query 的整体执行时间。
5. **speedup**：用于衡量多线程版本相对于单线程版本，或 x86 平台相对于 ARM 平台的加速效果。本文中加速比通常定义为
$$\text{Speedup} = \frac{T_{\text{baseline}}}{T_{\text{optimized}}}.$$
6. **profiling 指标**：包括 IPC、cache miss rate、branch misses 和 elapsed time 等。本文使用这些指标辅助解释不同算法的底层性能差异。

需要特别说明的是，本文中的 query latency 与 perf profiling 中的 elapsed time 采用不同计时口径。query latency 主要统计查询阶段的耗时，而 perf profiling 的 elapsed time 通常对应 full-program 运行时间，包含数据加载、索引构建和查询等多个阶段。因此，在结果分析中，二者不直接进行数值比较，而是分别用于评价查询性能和解释底层体系结构行为。

2 算法实现原理与并行优化

2.1 ANN 基础搜索与 SIMD 优化

在 lab2-SIMD 实验中，本文已经完成了基于 Flat scan 的向量化搜索实现[2]。Flat scan 对每个 query 遍历全部 base 向量，计算距离并维护 top-k 结果。该方法逻辑简单，且在不进行近似剪枝的情况下可以获得较高 Recall，因此可以作为后续索引优化和多线程优化的 baseline。

设 query 数量为 Q，base 向量数量为 N，向量维度为 d，则 Flat scan 的主要计算复杂度为

$$O(QNd).$$

[2]因此本节主要回顾 Flat-SIMD 的搜索流程和瓶颈，用于引出后续索引优化与多线程并行；其 SIMD 内核实现细节不再作为本实验重点展开。

在 DEEP100K 中，$N = 10^5$，$d = 96$，因此全量扫描需要大量距离计算。SIMD 优化主要降低单次距离计算的常数开销：在 ARM 平台上使用 NEON/ASIMD 指令，在 x86 平台上使用 AVX2/FMA 指令，一次处理多个 float 元素，从而提高内积或距离计算吞吐。

但是，SIMD 优化并不会减少需要访问的 base 向量数量。也就是说，Flat-SIMD 虽然加速了单次距离计算，但候选规模仍然是 N，整体 latency 仍然较高。

因此，在本实验中，Flat-SIMD 主要作为多线程并行化的基础 baseline，并进一步引出两类优化方向：一是通过 IVF、PQ、SQ 和 HNSW 等算法结构减少候选数量或降低距离估计成本；二是在 batch query 层面使用 Pthread 和 OpenMP 进行 query-level 并行。

2.2 IVF（Inverted File）

算法简介： IVF 通过聚类将向量空间划分为多个区域。首先对 base 集合 $X = \{x_i\}_{i=1}^{N} \subset \mathbb{R}^d$ 进行 k-means 聚类，得到 $nlist$ 个聚类中心 $\{c_j\}_{j=1}^{nlist}$，并将每个 base 向量分配到最近的中心所对应的倒排表中。

对于查询向量 q，先计算其与所有聚类中心的相似度：

$$s_j = \langle q, c_j \rangle, \quad j = 1, \ldots, nlist.$$

随后选取相似度最高的 $nprobe$ 个簇：

$$S = \mathrm{Top}_{nprobe}\{s_j\}.$$

最后仅在这些簇对应的倒排表中扫描候选向量，并维护 Top-k 结果。

Algorithm 1 IVF 查询过程

Input: 查询向量 q，聚类中心 $\{c_j\}_{j=1}^{nlist}$，倒排表 $\{L_j\}$，参数 $nprobe$，返回规模 k
Output: Top-k 近邻结果

```
初始化空结果堆 H
for j = 1 to nlist do
    计算 s_j = ⟨q, c_j⟩
end for
选取相似度最高的 nprobe 个簇，记为集合 S
for 每个簇 j ∈ S do
    for 每个候选向量 x_i ∈ L_j do
        计算 score = ⟨q, x_i⟩
        用 score 更新 Top-k 结果堆 H
    end for
end for
return H
```

设计动机： Flat scan 需要对每个 query 遍历全部 base 向量，而 IVF 只在与 query 最相关的若干个簇中搜索，从而显著减少候选数量。参数 $nprobe$ 控制搜索范围：$nprobe$ 越大，Recall 越高，但 latency 也会随之增加。

性能优化： IVF 的索引构建需要一次聚类，复杂度较高，但构建阶段通常不是查询性能分析的重点。查询阶段中，主要开销来自被选中倒排表内的候选扫描。由于不同倒排表长度可能不均匀，不同 query 的实际计算量也可能存在差异。

并行化点： 本文主要在 query 维度进行并行化，即不同线程处理不同 query。对于 OpenMP，还可以通过不同 schedule 策略缓解倒排表长度不均导致的负载不均。由于每个 query 独立维护 Top-k 结

果，query-level 并行不会改变搜索逻辑，因此 Recall 与串行版本保持一致。

2.3 IVF-PQ（Inverted File + Product Quantization）

算法简介： IVF-PQ 在 IVF 的基础上引入 Product Quantization，用压缩编码降低候选向量的距离估计成本。对于向量 $x \in \mathbb{R}^d$，将其划分为 m 个子向量：

$$x = [x^{(1)}, x^{(2)}, \ldots, x^{(m)}], \quad x^{(\ell)} \in \mathbb{R}^{d/m}.$$

在每个子空间内分别进行聚类，得到子空间码本。对于候选向量，其 PQ 编码可以表示为

$$x \approx [c_{b_1}^{(1)}, c_{b_2}^{(2)}, \ldots, c_{b_m}^{(m)}],$$

其中 b_ℓ 表示第 ℓ 个子空间对应的聚类中心编号。

查询时，先为 query 构建查找表：

$$LUT_{\ell,b} = \langle q^{(\ell)}, c_b^{(\ell)} \rangle, \quad \ell = 1, \ldots, m.$$

之后通过查表累加近似计算 query 与候选向量的相似度：

$$\langle q, x \rangle \approx \sum_{\ell=1}^{m} LUT_{\ell,b_\ell}.$$

Algorithm 2 IVF-PQ 查询过程

Input: 查询向量 q，IVF 倒排表 $\{L_j\}$，PQ 编码，码本，参数 $nprobe$、$rerank_p$、k
Output: Top-k 近邻结果

```
计算 q 与各 IVF 聚类中心的相似度
选取相似度最高的 nprobe 个簇，记为集合 S
根据 q 和 PQ 码本构建查找表 LUT
初始化候选堆 C
for 每个簇 j ∈ S do
    for 每个候选向量编码 code_i ∈ L_j do
        通过 LUT 查表累加，得到近似相似度 ŝ_i
        用 ŝ_i 更新前 rerank_p 个候选集合 C
    end for
end for
初始化结果堆 H
for 每个候选向量 x_i ∈ C do
    使用原始 float32 向量计算精确相似度 s_i = ⟨q, x_i⟩
    用 s_i 更新 Top-k 结果堆 H
end for
return H
```

设计动机： IVF 通过倒排表减少候选数量，PQ 进一步将候选扫描中的 float 距离计算转化为查表累加，从而降低计算量和存储开销。二者结合后，可以在较高 Recall 下获得较低查询延迟。

与 SIMD 的结合： IVF-PQ 中 LUT 构建和 rerank 阶段仍包含较多向量计算，可以继续使用 SIMD 加速。PQ 查表阶段虽然单次计算较轻，但访存更加随机。

并行化点： 本文主要对 IVF-PQ 的查询阶段进行 query-level 并行。不同线程分别处理不同 query，

独立完成 IVF 粗排、PQ 查表、候选筛选和 rerank。由于查询之间互不依赖，该方法同步开销较小；但当查表访问和候选维护成为瓶颈时，多线程加速可能不再线性增长。

2.4 HNSW 图索引方法

算法简介： HNSW（Hierarchical Navigable Small World）是一种基于多层图结构的近似最近邻搜索方法。其核心思想是将 base 向量组织为分层近邻图：高层图节点较少、连接较稀疏，用于快速确定搜索方向；底层图节点最多、连接更密集，用于进行精细搜索。查询时，算法从最高层入口点开始，通过贪心策略逐层向下寻找距离 query 更近的节点，最终在底层图中维护候选队列并返回 Top-k 结果。

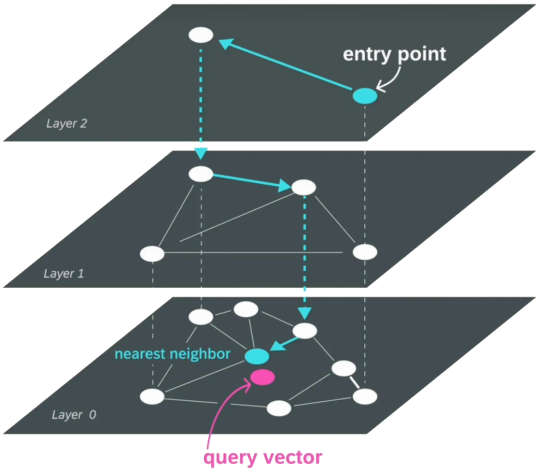


图 1: HNSW 多层图索引结构示意图

HNSW 的主要参数包括：

- M：每个节点的最大连接数，影响图的稠密程度和搜索路径质量；
- $efConstruction$：构建索引时的候选队列大小，影响索引质量和构建开销；
- $efSearch$：查询阶段的候选队列大小，影响 Recall 与 latency。

Algorithm 3 HNSW 查询过程

Input: 查询向量 q，HNSW 图索引 G，入口点 ep，参数 $efSearch$，返回规模 k
Output: Top-k 近邻结果

从最高层入口点 ep 开始
for 从最高层到第 1 层 **do**
    在当前层执行贪心搜索，更新距离 q 最近的入口点
end for
在底层图中初始化候选队列 C 和结果队列 R
while C 非空 **do**
    取出当前最优候选节点 v
    for 每个邻居节点 $u \in N(v)$ **do**
        if u 未访问且可能优于当前结果 **then**
            将 u 加入候选队列 C 和结果队列 R
        end if
    end for
    保持结果队列 R 的大小不超过 $efSearch$
end while
从 R 中返回 Top-k 结果

设计动机： Flat scan 需要遍历全部 base 向量，IVF 需要扫描被选中倒排表中的候选向量，而 HNSW 通过图上的导航式搜索逐步逼近近邻点，通常只需访问较少数量的节点即可得到高质量结果。因此，在索引构建完成后，HNSW 的 query latency 通常较低。

精度–效率权衡： HNSW 的查询性能主要由 $efSearch$ 控制。较大的 $efSearch$ 会扩大候选搜索范围，使 Recall 更稳定，但也会增加图遍历和候选队列维护开销。M 和 $efConstruction$ 更多影响索引构建质量：参数越大，图连接通常越充分，搜索精度越高，但 build time 和内存开销也会增加。

并行化点： HNSW 单个 query 内部的搜索过程存在较强数据依赖，下一步访问哪些节点取决于当前候选队列状态，因此不适合简单地进行细粒度并行化。本文主要采用 query-level 并行，即不同线程处理不同 query。该方式不改变 HNSW 原有搜索逻辑，可以降低 batch query 的 wall latency，但索引构建开销仍需要单独考虑。

2.5 多线程并行优化

并行化思路： 本文的 Pthread 与 OpenMP 版本均采用 query-level parallelism。由于不同 query 之间相互独立，每个 query 都可以独立完成搜索并维护自己的 Top-k 结果，因此可以将 query 集合划分给多个线程并行处理。在查询阶段，base 向量、IVF 倒排表、PQ 编码和 HNSW 图索引等结构均为只读共享数据；每个线程只写入自己负责 query 的结果区域，因此不需要复杂的锁同步。

设总 query 数为 Q，线程数为 T，则第 t 个线程处理的 query 区间为：

$$\left[\left\lfloor \frac{tQ}{T} \right\rfloor, \left\lfloor \frac{(t+1)Q}{T} \right\rfloor\right), \quad t = 0, 1, \ldots, T-1.$$

Algorithm 4 Query-level 多线程并行框架

Input: 查询集合 $Qset$，线程数 T，ANN 索引结构 $Index$
Output: 所有 query 的 Top-k 结果

```
将 Qset 按 query 编号划分为 T 个子区间
for 每个线程 t = 0, 1, ..., T − 1 in parallel do
    获取线程 t 对应的 query 区间 [l_t, r_t)
    for qid = l_t to r_t − 1 do
        读取 query q_qid
        在共享索引 Index 上执行搜索
        独立维护该 query 的 Top-k 结果
        将结果写入输出数组对应位置
    end for
end for
return 所有 query 的检索结果
```

Pthread 实现： Pthread 版本采用显式线程管理方式。程序手动创建线程，为每个线程传入线程编号、query 起止区间和共享索引指针，并在所有线程结束后通过 `pthread_join` 回收线程。该方式控制粒度较细，便于明确指定每个线程的工作范围，但代码相对冗长，需要手动管理线程生命周期。

OpenMP 实现： OpenMP 版本使用编译指导语句并行化 query 循环。其核心形式为：

```
#pragma omp parallel for schedule(...)
for (int qid = 0; qid < query_number; ++qid) {
    search_one_query(qid);
}
```

与 Pthread 相比，OpenMP 代码更加简洁，线程创建、任务划分和同步由 OpenMP runtime 自动完成。此外，OpenMP 可以方便地切换 `static`、`dynamic` 和 `guided` 等调度策略，因此适合分析不同任务划分方式对性能的影响。

小结： Pthread 与 OpenMP 在本文中采用相同的并行粒度，均不改变单个 query 的搜索逻辑，因此 Recall 理论上与对应串行版本保持一致。二者的主要区别在于 API 抽象层次和任务调度方式：Pthread 更底层、控制更显式；OpenMP 更简洁，并且更适合快速测试不同 schedule 策略。

3 实验设计

本章从实验问题出发设计测试方案，而不是单纯罗列运行命令。整体实验流程如图 2 所示：首先设计不同算法版本和并行 API，然后在 ARM/openEuler 与 x86/WSL2 平台上运行测试，记录 Recall@10、average latency、wall latency 等原始输出，最后整理为 CSV 数据并绘制图表进行分析。

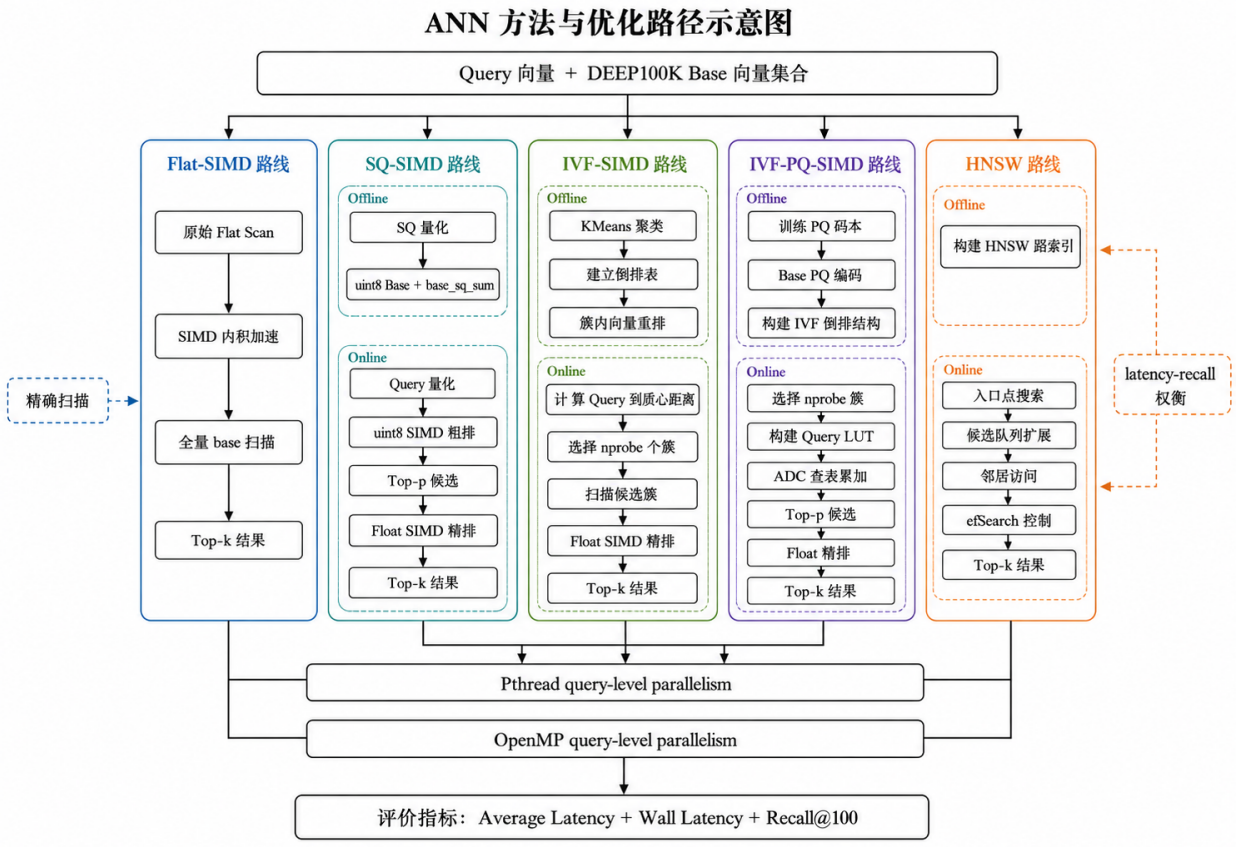


图 2: ANN 并行优化实验设计流程图。

为了保证实验设计具有针对性，本文将实验划分为算法综合对比、并行 API 对比、参数敏感性分析、OpenMP 调度策略分析、profiling 分析和跨平台对比六类。各类实验的控制变量、评价指标和实验目的如表 2 所示。

表 2: 实验设计总表

实验	数据来源	控制变量	评价指标	实验目的
综合性能对比	14 组 x86 查询结果	算法类型、API	Recall、wall latency	比较不同算法的总体查询性能，回答“谁最快”
Pthread/OpenMP 对比	同算法的两种 API 版本	API、线程数	wall latency、speedup	分析两种并行接口的性能差异
IVF 参数实验	IVF 参数网格	$nlist$、$nprobe$、API	Recall、wall latency	分析倒排参数对精度和延迟的影响
IVF-PQ 参数实验	IVF-PQ 参数网格	$nprobe$、$rerank_p$、API	Recall、wall latency	寻找 recall-latency 折中点
HNSW 参数实验	HNSW 参数网格	M、$efConstruction$、$efSearch$	Recall、wall latency、build time	分析图索引参数对查询性能的影响
OpenMP 调度实验	IVF-SIMD schedule 测试	static、dynamic、guided	wall latency	判断 schedule 是否影响负载均衡
profiling 分析	ARM/openEuler perf 结果	算法版本、API	IPC、cache miss rate、branch misses、elapsed time	从体系结构角度解释性能差异
跨平台对比	ARM 与 x86 查询结果	平台、算法、API	wall latency、speedup	分析架构差异对性能的影响

3.1 Flat-SIMD 多线程实验

Flat-SIMD 作为精确扫描 baseline，用于验证 query-level 并行在全量扫描场景下的基本加速效果。实验保持搜索逻辑不变，仅改变并行 API 和线程数，分别测试 Pthread 与 OpenMP 版本的查询性能。

该实验主要回答两个问题：第一，多线程能否在不影响 Recall 的情况下稳定降低 wall latency；第二，即使引入多线程，全量扫描是否仍然会受到候选规模过大的限制。

3.2 SQ-SIMD 多线程实验

SQ-SIMD 实验用于评估量化粗排加浮点精排结构在多线程环境下的表现。与 Flat-SIMD 相比，SQ 通过低精度量化向量进行粗排，减少原始 float32 精排候选数量，因此理论上可以降低查询延迟。

该实验控制变量包括并行 API、线程数以及候选规模相关参数，评价指标包括 Recall、wall latency 和跨平台 speedup。分析重点在于判断 SQ-SIMD 的瓶颈是否主要来自距离计算。如果 SQ 在 x86 上的加速幅度低于其他算法，说明其性能可能还受到候选维护、量化数据访问和内存行为的影响。

3.3 PQ-SIMD 多线程实验

PQ-SIMD 实验用于分析 Product Quantization 查询阶段及相关构建阶段的并行效果。PQ 通过查表累加近似距离，降低候选扫描阶段的计算成本；同时，PQ build 和 LUT 构建也可能成为额外开销，因此需要区分 query-only latency 和 full-program elapsed time。

该实验的控制变量包括并行 API、线程数，以及是否将 build/LUT 等阶段纳入计时。主要评价指标为 Recall、wall latency、build time 和 perf 指标。需要特别说明的是，query-only latency 主要反映查询阶段性能，而 perf elapsed time 通常包含数据加载、索引构建和查询等完整程序开销，二者不能直接混用。

3.4 IVF-SIMD 多线程实验

IVF-SIMD 实验用于分析倒排索引参数对查询性能的影响。本文测试不同 *nlist* 和 *nprobe* 组合下的 Recall 与 wall latency，并分别比较串行、Pthread 和 OpenMP 版本。

该实验主要回答两个问题: 第一, *nlist* 和 *nprobe* 如何影响 recall-latency trade-off; 第二, Pthread 与 OpenMP 是否会改变检索结果。由于多线程只改变 query 分配方式，不改变单个 query 的搜索逻辑，因此相同参数下 Recall 应保持一致，差异主要体现在 wall latency 上。

3.5 IVF-PQ-SIMD 多线程实验

IVF-PQ-SIMD 是本文重点分析的综合优化方法。它结合 IVF 的候选压缩能力和 PQ 的快速距离估计能力，并通过 float rerank 恢复精度。实验主要控制 *nprobe*、*rerank_p*、API 和线程数，观察不同参数组合下的 Recall 与 wall latency。

3.6 HNSW 多线程实验

HNSW 实验用于评估图索引方法的查询性能、构建成本和参数敏感性。控制变量包括 M、$efConstruction$、$efSearch$、并行 API 和线程数；评价指标包括 Recall、wall latency 和 build time。

该实验重点分析 $efSearch$ 对 Recall 与 latency 的影响。通常情况下，增大 $efSearch$ 可以提高搜索稳定性，但也会增加候选队列维护和图遍历开销。此外，HNSW 的 query latency 通常较低，但索引构建成本较高，因此需要与 IVF-PQ 等方法进行分场景比较。

3.7 参数敏感性与跨平台实验设计

参数敏感性实验用于解释不同算法中关键参数的选择依据。对于 IVF，本文主要测试 $nlist \times nprobe$ 参数网格；对于 IVF-PQ，测试 $nprobe \times rerank_p$ 参数组合；对于 HNSW，测试 M、$efConstruction$ 和 $efSearch$ 对 Recall 与 latency 的影响。这些实验用于说明参数变化如何影响候选规模、距离计算量和搜索精度。

跨平台实验用于比较 ARM/openEuler 与 x86/WSL2 两类平台上的查询性能差异。本文记录同一算法在两平台上的 wall latency，并计算 x86 相对 ARM 的 speedup。由于两平台在 CPU 架构、SIMD 指令集、缓存层次和编译器后端方面均存在差异，跨平台实验不只用于比较绝对速度，更用于分析不同算法的性能瓶颈是否与平台特征相关。

4 实验结果与分析

4.1 不同算法的综合性能对比

本节首先比较不同 ANN 方法在 x86/WSL2 平台上的综合查询性能。实验共包含 14 组方法，覆盖 Flat-SIMD、PQ-SIMD、SQ-SIMD、IVF-SIMD、IVF-PQ-SIMD 和 HNSW，并分别测试 OpenMP 与 Pthread 两种并行实现。评价指标主要采用 query-only wall latency，因为该指标反映多线程查询阶段的实际端到端耗时，更适合比较并行版本的查询性能。

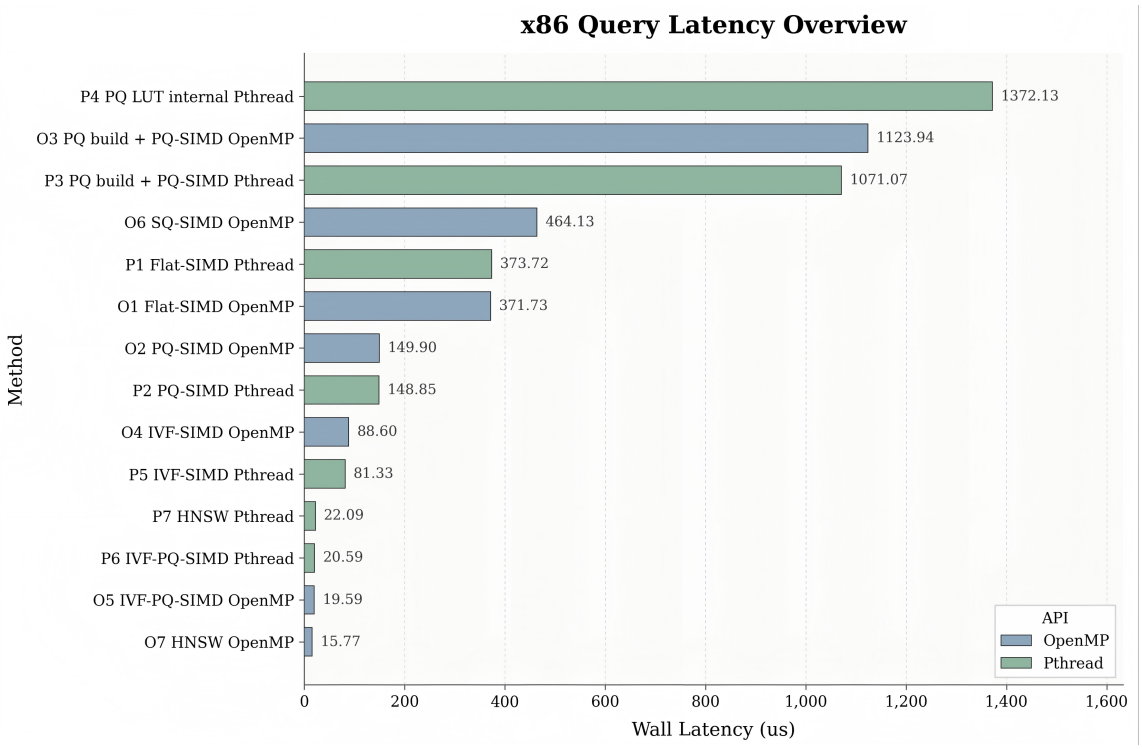


图 3: x86 平台 14 组方法的 query wall latency 总览。

从图 3 可以看出，不同算法之间的 latency 差异十分明显。Flat-SIMD 和 SQ-SIMD 虽然使用 SIMD 加速距离计算，但仍然需要扫描大量候选，因此查询延迟相对较高。相比之下，IVF-SIMD 通过倒排索引减少候选规模，latency 明显下降；IVF-PQ-SIMD 进一步利用 PQ 编码和查表累加降低候选扫描开销，在保持较高 Recall 的同时取得更低延迟。HNSW 则通过图索引进行导航式搜索，访问节点数量较少，因此查询阶段表现出最低的 latency。

表 3: x86 平台最快的代表方法

排名	方法	API	Recall@10	Wall latency / μs
1	HNSW	OpenMP	1.000000	15.77
2	IVF-PQ-SIMD	OpenMP	0.998850	19.59
3	IVF-PQ-SIMD	Pthread	0.998850	20.59
4	HNSW	Pthread	1.000000	22.09
5	IVF-SIMD	Pthread	0.959002	81.33
6	IVF-SIMD	OpenMP	0.959002	88.60

由表 3 可见，HNSW OpenMP 版本取得最低 wall latency，为 15.77 μs，同时 Recall@10 达到 1.0。IVF-PQ-SIMD OpenMP 和 Pthread 版本分别达到 19.59 μs 和 20.59 μs，Recall@10 为 0.998850，也处于高 Recall 区间。这说明在 x86 平台上，HNSW 和 IVF-PQ-SIMD 是查询阶段最具优势的两类方法。

不过，HNSW 的低查询延迟需要以较高的索引构建成本为代价[3]，而 IVF-PQ-SIMD 在查询延迟、Recall 和索引结构复杂度之间更加均衡。因此，从单次查询性能看，HNSW 最快；从综合实验方案看，IVF-PQ-SIMD 更适合作为本文后续 recall-latency trade-off 分析的主线方法。

[3]HNSW 的 build time 与 query latency 属于不同计时阶段。本文正文主要比较查询阶段 wall latency，索引构建成本作为补充因素用于解释 HNSW 的适用场景。

此外，OpenMP 与 Pthread 在同一算法下的差异通常小于不同算法结构之间的差异。例如 Flat-SIMD、PQ-SIMD 和 IVF-PQ-SIMD 的两种 API 结果较为接近，说明在 query-level 并行下，整体性能更多受到算法候选访问模式、距离计算方式和内存访问行为影响，而不是单纯由线程 API 决定。

4.2 Pthread 与 OpenMP 并行效果对比

本文中 Pthread 与 OpenMP 均采用 query-level 并行，即不同线程分别处理不同 query。因此，两者不会改变单个 query 内部的搜索过程，理论上 Recall 应与对应串行或同参数版本保持一致。二者的主要差异体现在 API 抽象层次、线程调度方式和运行时开销上：Pthread 需要手动创建线程并划分任务，OpenMP 则通过编译指导语句自动完成循环并行化和任务调度。

为了直接比较两种并行 API 的效果，本文选取 x86 平台上具有 OpenMP/Pthread 对应版本的代表算法进行对比，结果如表 4 所示。

表 4: x86 平台 OpenMP 与 Pthread query wall latency 对比

方法	OpenMP / μs	Pthread / μs	更快者
Flat-SIMD	371.73	373.72	OpenMP
PQ-SIMD	149.90	148.85	Pthread
IVF-SIMD	88.60	81.33	Pthread
IVF-PQ-SIMD	19.59	20.59	OpenMP
HNSW	15.77	22.09	OpenMP

从表 4 可以看出，OpenMP 与 Pthread 在多数算法上的性能差异并不大。Flat-SIMD、PQ-SIMD 和 IVF-PQ-SIMD 中，两种 API 的 wall latency 十分接近，说明在 query-level 并行粒度下，线程接口本身并不是决定性能的唯一因素。相比之下，不同算法结构之间的 latency 差异更加明显，例如 IVF-PQ-SIMD 和 HNSW 的查询延迟显著低于 Flat-SIMD 和 SQ-SIMD。

IVF-SIMD 中 Pthread 略快于 OpenMP，可能与手动划分 query 区间后调度开销较小有关；而 HNSW 中 OpenMP 明显快于 Pthread，说明对于图搜索这类单个 query 访问路径差异较大的算法，OpenMP runtime 的调度方式可能更有利于缓解负载不均。总体来看，Pthread 和 OpenMP 的性能差异小于算法选择带来的差异，因此本实验中优化的重点不应只放在线程 API 本身，而应结合算法结构、候选规模、访存模式和负载均衡共同分析。

需要强调的是，Pthread 与 OpenMP 只改变 batch query 的任务分配方式，不改变单个 query 的 ANN 搜索逻辑。因此，在相同算法参数下，两种并行实现的 Recall 基本一致，主要差异体现在 wall latency 和 total wall time 上。

为验证 query-level 并行的线程扩展性，本文补充测试了 Flat-SIMD 与 IVF-PQ-SIMD 在 1、2、4、8 线程下的 wall latency、speedup 和 efficiency，完整结果见附录表 10。

4.3 参数敏感性分析

4.3.1 IVF 参数敏感性

IVF 的核心参数为 *nlist* 和 *nprobe*。其中，*nlist* 表示倒排表数量，影响每个倒排表的平均长度；*nprobe* 表示查询时访问的倒排表数量，直接控制搜索范围。一般来说，*nprobe* 越大，访问候选越多，Recall 越高，但 wall latency 也随之上升。

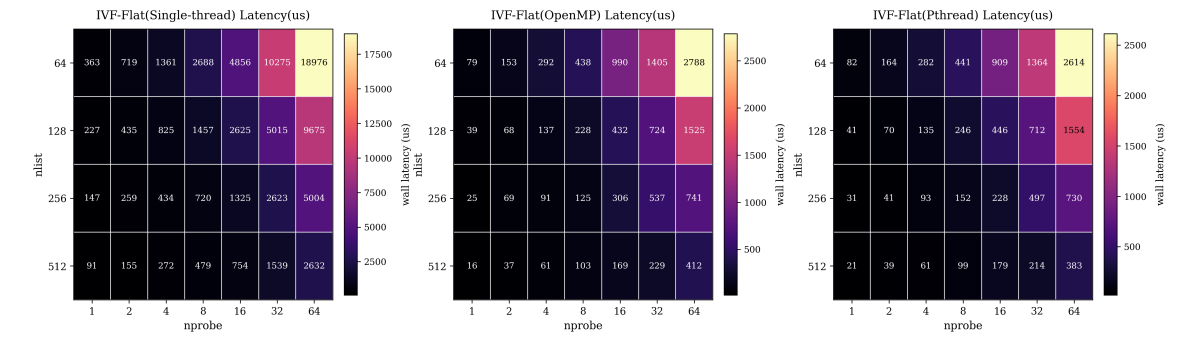


图 4: IVF_wall latency_heatmap

从图 4 可以看出，随着 *nprobe* 增大，三种方式查询延迟上升明显。这说明访问倒排表数量是影响 IVF 查询开销的关键因素。同时，在相同 *nprobe* 下，较大的 *nlist* 会缩短单个倒排表长度，降低候选扫描规模。

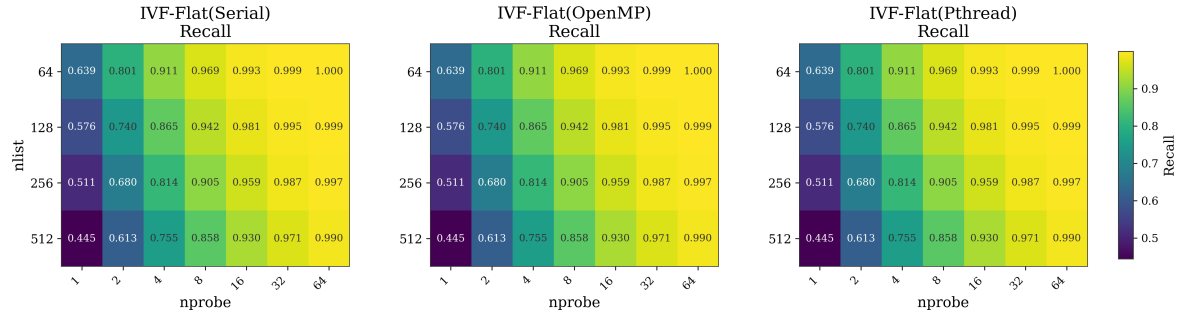


图 5: IVF_recall_heatmap

图 5 表明，Recall 主要由 *nlist* 与 *nprobe* 决定，而不是由 Pthread 或 OpenMP 决定。相同参数下，串行、OpenMP 和 Pthread 的 Recall 基本一致，说明多线程影响 wall latency，不改变搜索结果。

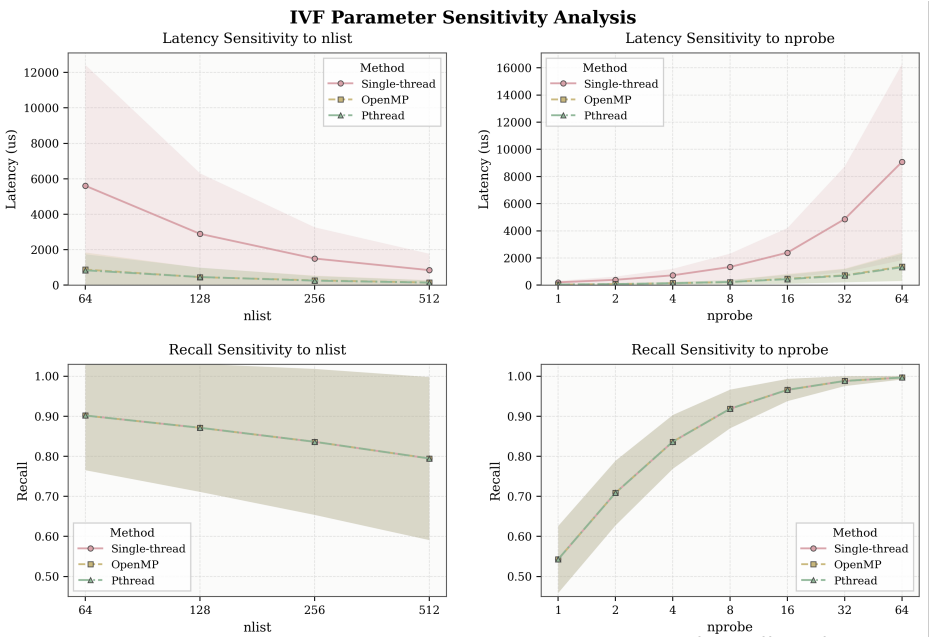


图 6: IVF 参数敏感性分析

图 6 对 IVF 参数趋势进行了汇总。增大 *nprobe* 能显著提高 Recall，但也会增加 latency；增大 *nlist* 可以减少单个倒排表长度，从而降低候选扫描开销，但在较小 *nprobe* 下可能导致搜索覆盖不足。因此，IVF 的参数选择需要在搜索范围和候选规模之间折中。

表 5: IVF 参数影响总结

参数	增大后的主要影响	典型 trade-off
nlist	倒排表数量增加，单表候选数减少	latency 可能下降，但小 *nprobe* 下 Recall 可能受影响
nprobe	查询访问更多倒排表	Recall 上升，但 latency 增加
API	改变 query 任务分配方式	不改变 Recall，主要影响 wall latency

4.3.2 IVF-PQ 参数敏感性

IVF-PQ 的核心参数为 *nprobe* 和 *rerank_p*。其中，*nprobe* 控制 IVF 阶段访问的倒排表数量，*rerank_p* 控制 PQ 粗排后进入 float32 精排的候选数量。二者共同决定 Recall 与 latency 的折中关系。

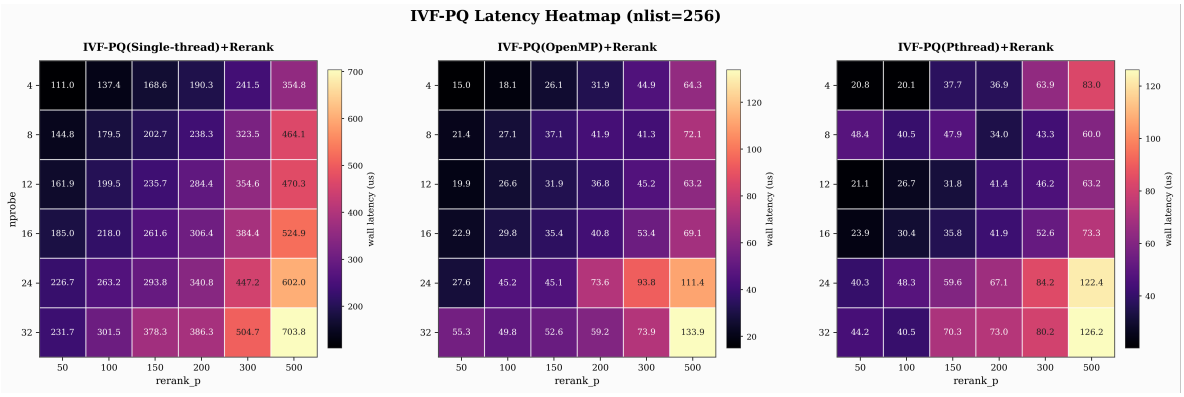


图 7: IVF-PQ_wall latency_heatmap

图 7 展示了相同参数组合下的 latency 代价。总体来看，*nprobe* 和 *rerank_p* 增大都会提高查询延迟：前者增加访问的倒排表数量，后者增加 float32 精排候选规模。因此，IVF-PQ 的参数选择不能只追求最高 Recall，而应关注高 Recall 区间内的最低 latency。

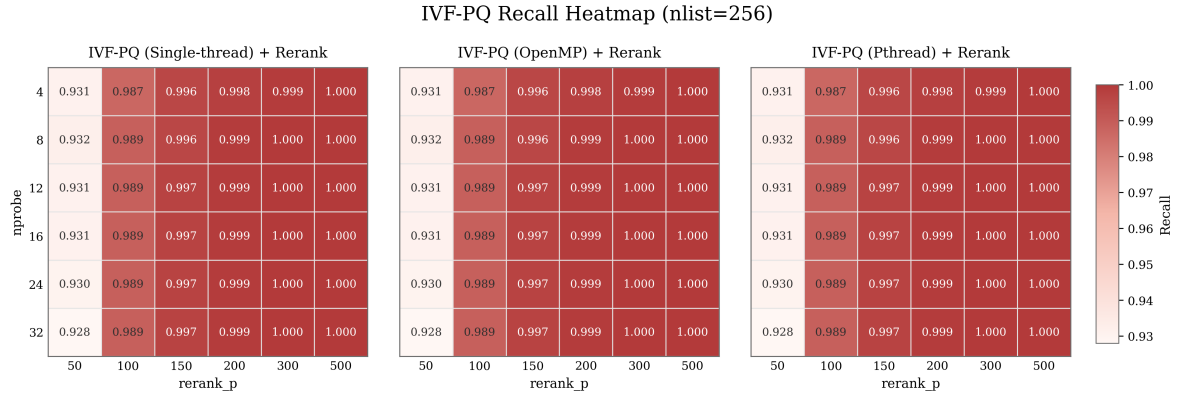


图 8: IVF-PQ_recall_heatmap

从图 8 可以看出，*rerank_p* 对 Recall 的影响非常明显。当 *rerank_p* 较小时，PQ 粗排保留的

候选不足，容易遗漏真实近邻；随着 *rerank_p* 增大，进入 float32 精排的候选增多，Recall 迅速提升并趋近于 1.0。

表 6: IVF-PQ 参数影响总结

参数	增大后的主要影响	典型 trade-off
nprobe	访问更多倒排表，候选覆盖更充分	Recall 上升，但 PQ 扫描开销增加
rerank_p	更多候选进入 float32 精排	Recall 上升，但 rerank latency 增加
PQ 查表	降低候选距离估计成本	可能引入量化误差和随机访存开销

此外，P3 和 P4 等阶段性版本用于观察 PQ build 与 LUT internal 阶段的开销。结果表明，IVF-PQ 的性能不仅受 query-level 并行影响，还与 LUT 构建、查表访问和候选维护有关。因此，后续若进一步实现 list-level 或 rerank-level 并行，还需要重点处理局部 Top-k 合并和随机访存带来的额外开销。

4.3.3 HNSW 参数敏感性

HNSW 的主要参数包括 M、$efConstruction$ 和 $efSearch$。其中，M 和 $efConstruction$ 主要影响索引构建质量和构建成本，$efSearch$ 直接影响查询阶段的候选搜索范围。本文重点观察 $efSearch$ 对 Recall 和 wall latency 的影响，并通过参数 pairplot 辅助分析不同参数之间的关系。

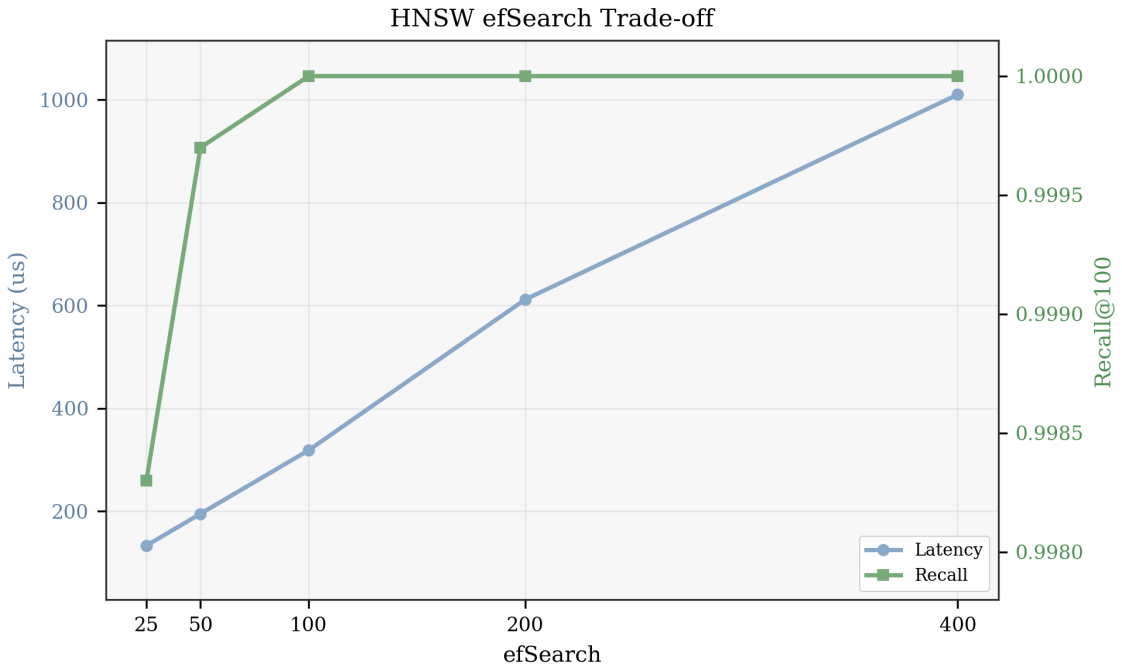


图 9: HNSW 在不同 $efSearch$ 下的 latency-recall trade-off。

图 9 展示了 $efSearch$ 变化对 HNSW 查询性能的影响。可以看到，随着 $efSearch$ 增大，HNSW 查询需要维护更大的候选队列并访问更多邻居节点，因此 latency 明显上升。同时，Recall 在较小 $efSearch$ 时已经接近 1.0，继续增大 $efSearch$ 带来的精度收益较小，边际收益逐渐降低。

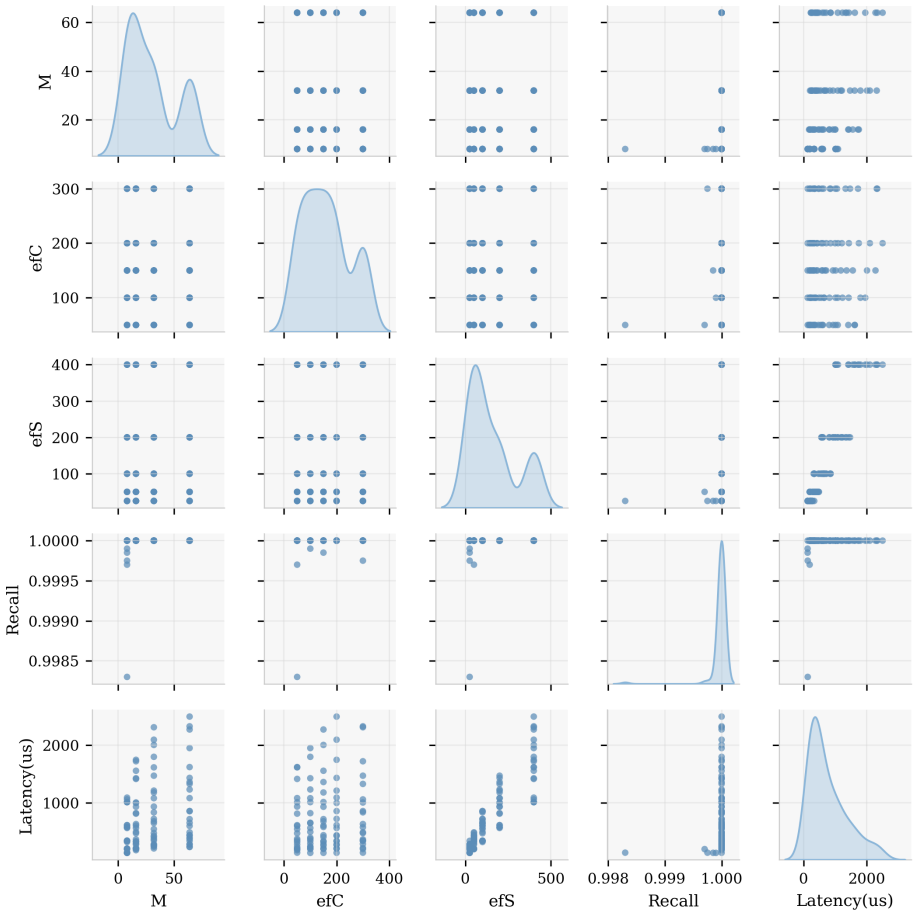


图 10: HNSW 参数分析

图 10 从多参数角度展示了 HNSW 的性能变化。整体来看，$efSearch$ 与 latency 的正相关关系较为明显；M 和 $efConstruction$ 更多影响图结构质量和构建成本。由于 HNSW 的 Recall 在多数参数组合下已经接近 1.0，因此在本数据集上继续增大搜索参数的主要代价体现为 latency 和 build time 增加。

表 7: HNSW 参数影响总结

参数	增大后的主要影响	典型 trade-off
M	图连接更稠密，搜索路径更充分	Recall 可能更稳定，但内存和构建成本增加
$efConstruction$	构建时候选范围更大	索引质量提高，但 build time 增加
$efSearch$	查询时访问更多候选节点	Recall 更稳定，但 latency 增加

综上，IVF、IVF-PQ 和 HNSW 的参数敏感性体现了不同算法的优化机制。IVF 主要通过 $nprobe$ 控制搜索范围；IVF-PQ 通过 $nprobe$ 与 $rerank_p$ 共同控制候选覆盖和精排规模；HNSW 则通过 $efSearch$ 控制图搜索的候选队列大小。参数选择的核心并不是单独追求最高 Recall 或最低 latency，而是在满足精度要求的前提下选择更低延迟的配置。

4.4 OpenMP 调度策略分析

OpenMP 版本通过 `parallel for` 对 query 循环进行并行化。除了线程数以外，`schedule` 策略也会影响任务划分方式。对于 Flat-SIMD 这类每个 query 计算量较均匀的算法，静态划分通常已经能够取得较稳定的效果；但对于 IVF-SIMD，different query 可能访问不同长度的倒排表，因此每个 query 的实际扫描候选数并不完全相同，容易出现线程负载不均。

本文选取 IVF-SIMD 作为 OpenMP 调度策略测试对象，比较 `static`、`dynamic,1`、`dynamic,8` 和 `guided,8` 四种策略。实验结果如图 11 所示。

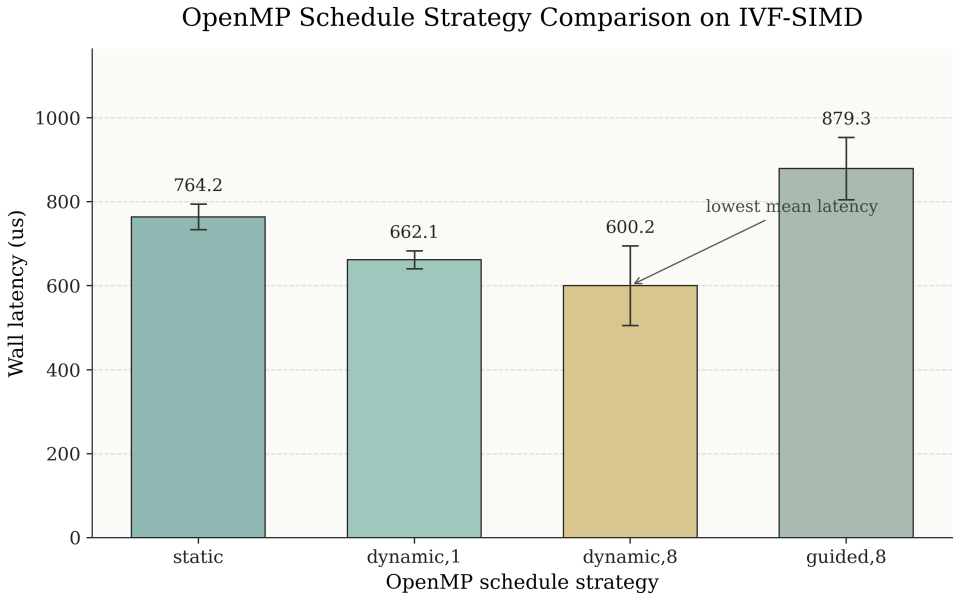


图 11: IVF-SIMD 中不同 OpenMP schedule 策略的 wall latency 对比。

表 8: IVF-SIMD OpenMP schedule 对比结果

Schedule	Avg latency / μs	Wall latency / μs
`static`	5200.82	764.20
`dynamic,1`	5061.19	662.08
`dynamic,8`	4537.48	600.19
`guided,8`	6455.09	879.30

从图 11 和表 8 可以看出，`dynamic,8` 取得了最低的 wall latency，为 600.19 μs，优于 `static` 的 764.20 μs。这说明在 IVF-SIMD 中，适当的动态调度能够缓解倒排表长度不均造成的负载不均，使空闲线程能够继续领取新的 query 任务。

不过，调度策略并不是越动态越好。`dynamic,1` 虽然也改善了负载均衡，但由于任务块过小，调度次数更多，额外运行时开销更高，因此性能不如 `dynamic,8`。`guided,8` 在本实验中表现最差，说明 guided 策略的任务块递减机制并不一定适合当前 IVF 查询负载。综合来看，OpenMP schedule 的最优选择取决于任务粒度、单个 query 的计算量差异以及运行时调度开销之间的平衡。

4.5 Profiling 与体系结构分析

除 query-only latency 外，本文还在 ARM/openEuler 平台使用 `perf stat` 进行 full-program profiling，统计 IPC、cache miss rate、branch misses 和 elapsed time 等指标，用于辅助解释不同算法的底层性能差异。需要注意的是，`perf stat` 的 elapsed time 通常包含数据加载、索引构建和查询等完整程序阶段，而前文的 query latency 只统计查询阶段耗时。因此，本文不将二者直接数值比较，而仅将 profiling 结果作为体系结构层面的辅助分析。

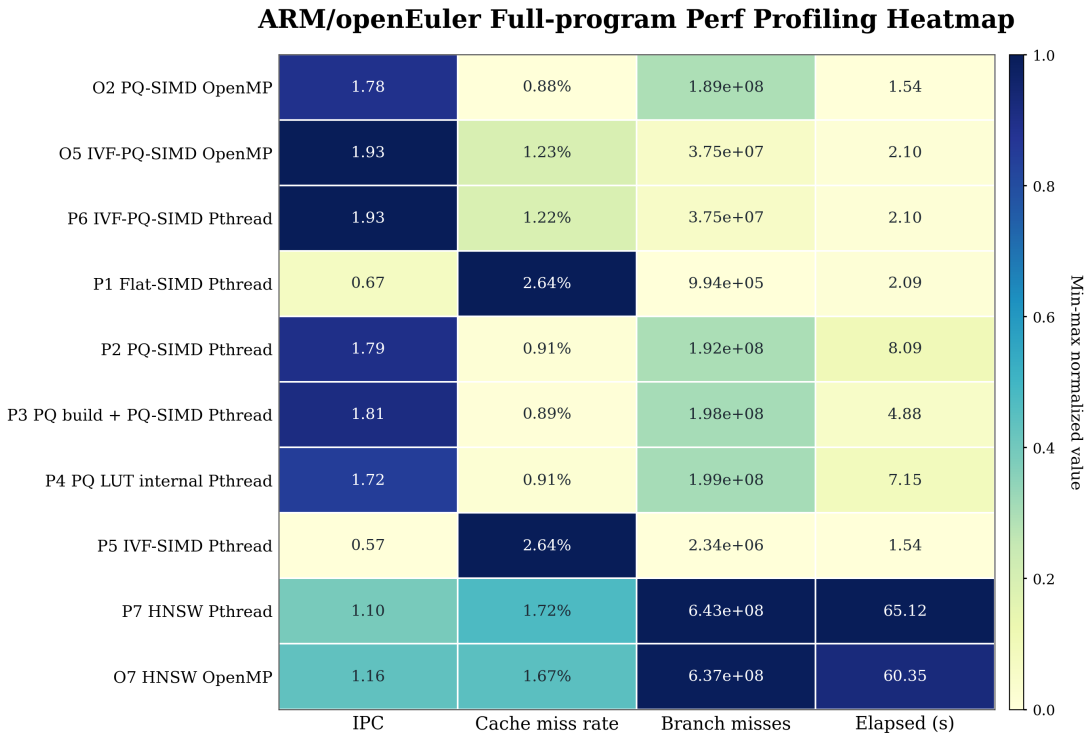


图 12: perf profiling_heatmap

从图 12 可以看出，不同 ANN 方法具有不同的底层性能特征。PQ 和 IVF-PQ 在减少距离计算量的同时引入查表和随机访存，cache miss 与内存访问可能成为新的限制因素；HNSW 查询阶段访问节点较少，query latency 较低，但 full-program 表现仍会受到索引构建阶段影响。

在 x86/WSL2 平台上，本文尝试使用 `perf` 采集硬件性能计数器，但当前环境下部分硬件事件不支持显示，因此 x86 平台主要用于 query latency 和跨平台加速比测试。总体来看，profiling 结果说明，多线程加速效果不能只由线程数解释，还需要结合算法访存模式、候选规模和索引结构共同分析。

4.6 ARM 与 x86 跨平台对比分析

为分析平台架构对 ANN 查询性能的影响，本文对 ARM/openEuler 与 x86/WSL2 两个平台上的代表方法进行比较。ARM 平台采用 Kunpeng-920 处理器和 NEON/ASIMD 指令集，x86 平台采用 Intel Core i9-14900HX 处理器和 AVX2/FMA 指令集。[4]

[4]由于两类平台在核心结构、缓存层次、SIMD 宽度和编译器后端等方面均存在差异，因此本节主要关注相对加速比，而非绝对时间的严格公平比较。

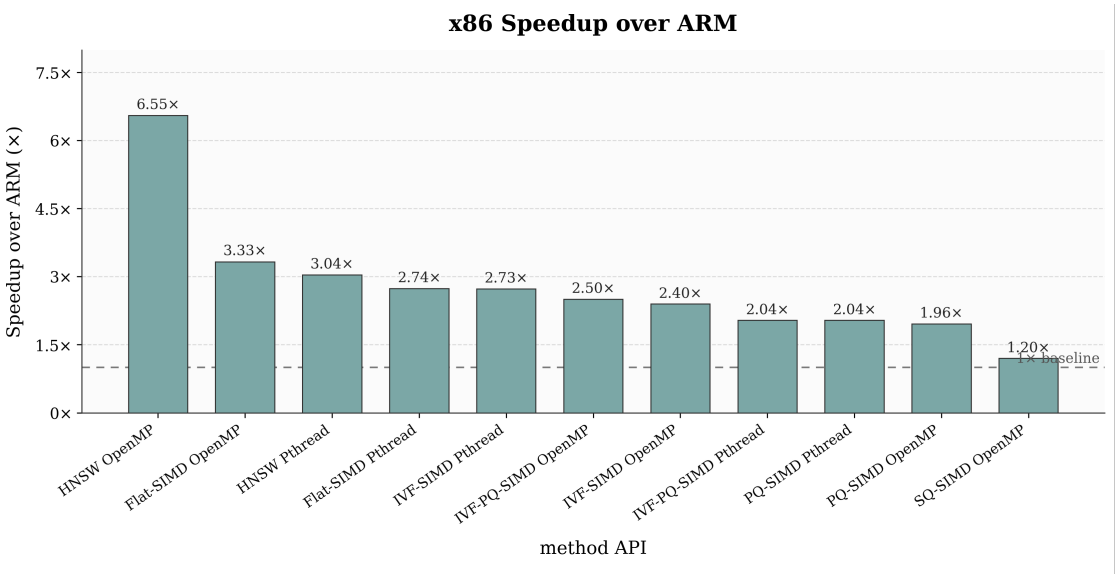


图 13: x86/WSL2 相对 ARM/openEuler 的加速比。

表 9: ARM/openEuler 与 x86/WSL2 平台 query wall latency 对比

方法	API	ARM / μs	x86 / μs	x86 相对 ARM 加速
Flat-SIMD	OpenMP	1236.55	371.73	3.33×
Flat-SIMD	Pthread	1023.54	373.72	2.74×
PQ-SIMD	OpenMP	293.61	149.90	1.96×
PQ-SIMD	Pthread	303.40	148.85	2.04×
IVF-SIMD	OpenMP	212.37	88.60	2.40×
IVF-SIMD	Pthread	222.03	81.33	2.73×
IVF-PQ-SIMD	OpenMP	48.96	19.59	2.50×
IVF-PQ-SIMD	Pthread	42.00	20.59	2.04×
SQ-SIMD	OpenMP	554.95	464.13	1.20×
HNSW	OpenMP	103.29	15.77	6.55×
HNSW	Pthread	67.17	22.09	3.04×

从图 13 和表 9 可以看出，x86 平台在多数方法上明显快于 ARM 平台。其中 HNSW OpenMP 的加速比最高，达到 6.55×[5]；Flat-SIMD、IVF-SIMD 和 IVF-PQ-SIMD 也均获得了约 2× 以上的加速。这说明更强的单核性能、SIMD 指令支持和缓存层次对 ANN 查询具有明显影响。

同时，不同算法的跨平台加速幅度并不一致。Flat-SIMD 和 IVF-PQ-SIMD 能较好受益于 x86 的计算能力，而 SQ-SIMD OpenMP 的加速比仅为 1.20×，说明其瓶颈可能更多来自量化扫描、候选维护或内存访问。总体来看，跨平台实验表明 ANN 性能不仅取决于并行 API，也与算法结构和底层体系结构密切相关。

[5]该加速比仅针对查询阶段的 query-only wall latency，不包含 HNSW 索引构建时间；HNSW 的构建成本在前文 profiling 与参数分析中单独讨论。

5 总结

5.1 实验结论

本实验围绕 DEEP100K 数据集上的 ANN 查询任务，在已有 SIMD 优化基础上进一步实现并比较了 Pthread 与 OpenMP 两类多线程并行方法。主要结论如下：

1. **算法结构对性能影响最大**。 Flat-SIMD 虽然 Recall 较高，但全量扫描导致 latency 较大；IVF、PQ、SQ 和 HNSW 通过减少候选规模、降低距离估计成本或改变搜索结构，有效降低了查询开销。其中，IVF-PQ-SIMD 在速度和精度之间较为均衡，HNSW 查询阶段最快，但索引构建成本较高。

2. **Pthread 与 OpenMP 整体性能接近**。在 query-level 并行粒度下，两者都将不同 query 分配给不同线程处理，不改变单个 query 的搜索逻辑，因此相同参数下 Recall 基本一致。实验结果表明，线程 API 的差异通常小于算法结构本身带来的差异。

3. **关键参数体现 recall-latency 权衡**。 IVF 中的 $nprobe$、IVF-PQ 中的 $rerank_p$ 以及 HNSW 中的 $efSearch$ 增大时，通常能提高 Recall 或搜索稳定性，但也会增加候选扫描、精排或图遍历开销。因此参数选择需要在精度和延迟之间折中。

4. **跨平台性能与体系结构相关**。x86/WSL2 平台在多数方法上快于 ARM/openEuler 平台，但不同算法的加速幅度不同，说明 ANN 查询性能还受到 SIMD 指令集、缓存层次、访存模式和算法结构的共同影响。

5.2 不足与改进方向

本实验仍有以下不足：

1. 当前主要采用 query-level 并行，对单个 query 内部的细粒度并行探索不足。后续可以继续研究 IVF 倒排表内部扫描、PQ 查表累加、rerank 阶段以及 HNSW 图搜索过程中的并行优化。

2. profiling 主要采用 full-program 口径，可能同时包含数据加载、索引构建和查询阶段。后续可以进一步拆分 build、query、rerank 等阶段，分别统计性能指标，以便更准确地定位瓶颈。

5.3 个人心得体会

通过本次实验，我对并行优化有了更具体的认识。并行编程并不是简单地增加线程数，也不是只比较 Pthread 和 OpenMP 哪个接口更快，而是需要结合算法结构、任务粒度、访存模式和平台特征综合分析。尤其是在 ANN 查询任务中，候选规模和索引结构往往比线程 API 本身更能决定性能上限。

实验过程中，我也体会到数据整理和计时口径统一的重要性。average latency、wall latency、build time 和 perf elapsed time 对应不同的统计范围，如果混在一起比较，很容易得到错误结论。因此，保留日志、整理 CSV、绘制图表并统一分析口径，也是性能实验中非常重要的一部分。

附录 A 线程扩展性实验数据

为补充分析线程数与性能之间的关系，本文选取 Flat-SIMD 和 IVF-PQ-SIMD 两类代表方法，在 ARM/openEuler 平台上分别测试 1、2、4、8 线程下的查询性能。Speedup 和 parallel efficiency 定义如下：

$$Speedup(p) = \frac{T_1}{T_p}, \qquad Efficiency(p) = \frac{Speedup(p)}{p}.$$

其中，T_p 表示 p 个线程下的 query-only wall latency。表 10 中结果由三次重复实验取平均得到。

表 10: 代表方法的 1/2/4/8 线程扩展性实验结果

Method	API	Threads	Wall Lat. / μs	Speedup	Efficiency	Recall@10
Flat-SIMD	OpenMP	1	8461.74	1.00	1.00	0.999950
Flat-SIMD	OpenMP	2	4048.26	2.09	1.05	0.999950
Flat-SIMD	OpenMP	4	2209.17	3.83	0.96	0.999950
Flat-SIMD	OpenMP	8	1235.52	6.85	0.86	0.999950
Flat-SIMD	Pthread	1	6714.98	1.00	1.00	0.999950
Flat-SIMD	Pthread	2	4277.82	1.57	0.78	0.999950
Flat-SIMD	Pthread	4	2266.10	2.96	0.74	0.999950
Flat-SIMD	Pthread	8	1125.90	5.96	0.75	0.999950
IVF-PQ-SIMD	OpenMP	1	303.23	1.00	1.00	0.998850
IVF-PQ-SIMD	OpenMP	2	154.78	1.96	0.98	0.998850
IVF-PQ-SIMD	OpenMP	4	78.52	3.86	0.97	0.998850
IVF-PQ-SIMD	OpenMP	8	46.60	6.51	0.81	0.998850
IVF-PQ-SIMD	Pthread	1	305.83	1.00	1.00	0.998850
IVF-PQ-SIMD	Pthread	2	153.25	2.00	1.00	0.998850
IVF-PQ-SIMD	Pthread	4	78.19	3.91	0.98	0.998850
IVF-PQ-SIMD	Pthread	8	58.65	5.21	0.65	0.998850

从表 10 可以看出，随着线程数从 1 增加到 8，Flat-SIMD 与 IVF-PQ-SIMD 的 wall latency 均明显下降，说明 query-level parallelism 能够有效提升批量查询吞吐。同时，不同线程数下 Recall 保持不变，说明该并行方式只改变 query 的任务分配，不改变单个 query 的搜索逻辑。8 线程下 efficiency 有所下降，主要与线程调度、访存竞争和任务粒度有关。

附录 B 原始实验数据

ID	Method	API	Threads	Runs	Recall@10	Avg Lat. / μs	Wall Lat. / μs	Total Wall / μs	ARM Wall / μs	x86 Wall / μs	x86/ARM
O1	Flat-SIMD	OpenMP	8	3	0.999950	2921.64	371.73	743457	1236.55	371.73	3.33×
O2	PQ-SIMD	OpenMP	8	3	0.956053	1174.39	149.90	299790	293.61	149.90	1.96×
O3	PQ build + PQ-SIMD	OpenMP	1	3	0.956053	1122.35	1123.94	2247890	2193.31	1123.94	1.95×
O4	IVF-SIMD	OpenMP	8	3	0.959002	672.13	88.60	177191	212.37	88.60	2.40×
O5	IVF-PQ-SIMD	OpenMP	8	3	0.998850	143.55	19.59	39183	48.96	19.59	2.50×
O6	SQ-SIMD	OpenMP	8	3	0.999950	3629.98	464.13	928254	554.95	464.13	1.20×
O7	HNSW	OpenMP	8	3	1.000000	79.83	15.77	31545	103.29	15.77	6.55×
P1	Flat-SIMD	Pthread	8	3	0.999950	2922.05	373.72	747439	1023.54	373.72	2.74×
P2	PQ-SIMD	Pthread	8	3	0.956053	1155.28	148.85	297704	303.40	148.85	2.04×
P3	PQ build + PQ-SIMD	Pthread	1	3	0.956053	1069.55	1071.07	2142130	2234.36	1071.07	2.09×
P4	PQ LUT internal	Pthread	1	3	0.956053	1370.26	1372.13	2744260	2866.46	1372.13	2.09×
P5	IVF-SIMD	Pthread	8	3	0.959002	604.82	81.33	162652	222.03	81.33	2.73×
P6	IVF-PQ-SIMD	Pthread	8	3	0.998850	143.60	20.59	41178	42.00	20.59	2.04×
P7	HNSW	Pthread	8	3	1.000000	141.36	22.09	44190	67.17	22.09	3.04×

表 11: 各方法不同实现的性能表现原始数据